

PREFLIGHT — COMPLETE PRODUCT SPECIFICATION v1.2

NMIT Hacks 2026 · AI & ML Track · 48-Hour Hackathon

Last updated: May 8, 2026

WHAT IS PREFLIGHT?

Preflight is a behavioral pre-execution interceptor that lives natively in the GitHub PR lifecycle. It detects novel malicious npm packages through code analysis and AI reasoning — not CVE lookups — and gets smarter with every scan through community threat intelligence.

One attack surface. One ecosystem. One moment in time.
The PR. Before merge. Before execution. That is where we live.

Preflight is THREE things with completely different user experiences:

THING 1 — A GitHub Action (the actual product)

Developers add one line of YAML to their existing workflow file.

GitHub fetches it from our repo via HTTPS + SHA pinning.

No download. No install. No sign-up. 30 seconds to protect a repo forever.

- name: Preflight Supply Chain Scan

uses: preflight-ai/preflight@v1

with:

lockfile: package-lock.json

Why NOT npm: A supply chain security tool distributed via npm is a logical contradiction. We live outside the ecosystem we protect.

THING 2 — A Cloud Analysis API (invisible to the user)

FastAPI service on Render. The GitHub Action calls it automatically.

Users never interact with it directly. It is a pure implementation detail.

URL: <https://preflight-api.onrender.com>

THING 3 — A Website (for discovery and community intelligence)

preflight.dev — public, no account required.

Landing page, /demo, /dashboard, /scans/:id, /packages/:name

Visit, run the demo, copy the install snippet. No login. No payment.

THE COMPLETE USER JOURNEY IN ONE PARAGRAPH:

A developer is on GitHub. Someone opens a PR bumping axios 1.7.9 → 1.7.10.
Preflight's workflow is already in the repo (one line, added once).
It triggers automatically. 30 seconds later a comment appears on the PR.
BLOCK — 94% confidence. PR status check turns red.
The developer reads Gemini's plain-English explanation. Does not merge.
They avoided a RAT. They never opened a terminal. Never visited our site.
Never signed up for anything. That is the product.

WHAT PREFLIGHT IS NOT

Read this before building anything. Every boundary here is a deliberate product decision, not a limitation. Knowing what we are not makes every build decision faster and every pitch answer sharper.

--- NOT BY DISTRIBUTION ---

NOT an npm package.

The most important one. Installing a supply chain security tool via the supply chain it protects is a logical contradiction. If we shipped as "npm install -g preflight", we'd be asking users to trust npm to protect them from npm. We are distributed as a GitHub Action — fetched from GitHub's own infrastructure over HTTPS with SHA pinning. Outside the attack surface entirely. This is non-negotiable forever.

NOT a CLI tool.

Preflight has no terminal presence on a developer's machine. No brew install. No PATH entry. No local binary. It runs in GitHub's cloud, triggered by a PR.

NOT a browser extension, desktop app, or VS Code plugin.

Zero local presence on the developer's machine. Zero surface area to compromise.

--- NOT BY WHAT IT SCANS ---

NOT a CVE scanner.

This is the sharpest product line. npm audit, Snyk, and Dependabot all check CVE databases. They all missed the axios attack because it had no CVE — it was a novel, legitimately-signed payload. Preflight does not check CVE databases.

It never will. That is someone else's solved problem.

We detect behaviourally novel threats. Packages with no prior record of malice.

NOT a source code scanner.

Preflight never touches your application code. Not your .ts files, not your business logic, not your API routes. It only looks at the pre/postinstall scripts of npm packages being introduced or upgraded in the PR diff.

NOT a secrets scanner.

Finding leaked API keys or credentials in your repo is a different tool's job.

GitHub Secret Scanning does this. Not Preflight.

NOT a license checker.

Whether a dependency is MIT or GPL is irrelevant to Preflight entirely.

NOT a transitive dependency scanner.

Preflight only analyses packages that changed in the PR diff — direct bumps.

It does not crawl the entire dependency tree of your project. That scope would make the tool slow, noisy, and unshippable in 48 hours.

One PR. Changed packages only. Direct dependencies only.

NOT a Python / Ruby / Go / Rust / any-other-ecosystem tool.

npm only. This is not a limitation — it is deliberate focus.

The axios attack was npm. The ecosystem with the most supply chain attacks is npm. We go deep on one thing, not shallow on everything.

Breadth can come later. Depth wins the hackathon and protects real users now.

--- NOT BY WHAT IT DOES AFTER DETECTION ---

NOT an auto-fixer.

Preflight blocks and explains. It does not automatically roll back the dependency, open a counter-PR, pin a version, or modify your lockfile.

The human makes the final call. Always. We give the verdict and the reasoning.

The developer decides what to do with it.

NOT an incident response tool.

Preflight fires before the code runs. If a malicious package has already been merged and executed on production — that is outside our scope entirely.

We are pre-execution only. Post-merge incident response is a different category of tooling with a different threat model.

NOT a real-time production monitor.

We do not watch your running application. We do not monitor outbound network

calls from your servers. We do not instrument your runtime. We live entirely in the PR lifecycle — before merge, before deploy, before execution.

--- NOT BY WHAT IT EXECUTES ---

NOT a sandbox that runs postinstall scripts.

This is critical for safety. We analyse postinstall scripts through AST parsing and diffing. We never execute them. A security tool that runs suspicious scripts to observe behaviour would itself become an attack vector. We read the code. We reason about it. We never run it.

--- NOT BY BUSINESS MODEL ---

NOT a SaaS.

No accounts. No login. No org dashboard. No pricing tiers. No "upgrade to Pro." No enterprise contracts. No data sold. The GitHub Action is free and open source forever. The website is a public utility, not a product you pay for.

NOT an enterprise security platform.

Socket.dev is that. We are not competing with them on their terms. We are the open-source, zero-friction, zero-cost alternative that any solo developer can add in 30 seconds. The product we are building is for everyone, not a procurement decision.

--- NOT BY DATA ---

NOT storing anything sensitive about your repo.

MongoDB stores: package name, versions, verdict, signals, confidence, duration. That is all. We do not store your source code, your package-lock.json contents, your organisation name (repo field is optional and anonymisable), or any secrets. No PII. No proprietary code. No credentials.

NOT a CVE database.

We generate novel threat intelligence from behavioural analysis. We are not ingesting, storing, or republishing existing CVE data from any source.

--- THE BOUNDARY IN ONE SENTENCE ---

Preflight does exactly one thing: intercept npm dependency changes on GitHub pull requests, analyse the behaviour of the new code before it runs, and post a plain-English verdict powered by Gemini AI — before a single line of malicious code ever executes on anyone's machine.

Everything outside that sentence is not Preflight.

THE REAL INCIDENT (anchor for every demo and pitch)

March 31, 2026 — 00:21 UTC

A North Korean state actor (Sapphire Sleet) hijacked the axios npm account.

axios: 70M+ weekly downloads. RAT was live for 3 hours.

npm audit missed it. Snyk missed it. Dependabot missed it.

The package was legitimately signed. No CVE. No prior record.

Preflight would have blocked it in under 30 seconds.

TIER 1 SCOPE (What we build and ship at NMIT Hacks)

1. GitHub Action — triggers on every PR that modifies package-lock.json
 2. Preflight Analysis API — 4-signal engine deployed on Render
 3. Gemini AI — LLM reasoning layer (verdict synthesis)
 4. MongoDB Atlas — community threat intelligence database
 5. Web Dashboard — landing page + live scan feed + demo trigger
-
-

HOW THE 3 PARTS TALK TO EACH OTHER

GitHub (PR) → GitHub Action → POST /analyze → FastAPI on Render



4 signal layers run



Write result to MongoDB



GitHub Action ← Response (verdict + signals)

↓
Post PR comment on GitHub
Set status check (pass / fail)

Website (preflight.dev) → GET /scans, /packages → FastAPI → MongoDB
/demo page → POST /analyze → same API

DETECTION ENGINE — 4 SIGNAL LAYERS

SIGNAL 1 — Script Diff Analysis

Input: package name, old version, new version

Process: Fetch both tarballs from npm registry.

Extract package.json scripts: preinstall, postinstall, install.

Diff scripts between versions.

Flags:

- New hook added (not in old version) → HIGH
- Existing hook modified → MEDIUM
- Hook removed → LOW (log only)

Output: { flagged, new_hooks, changed_hooks, reason }

SIGNAL 2 — AST Behavioral Scan

Input: postinstall / preinstall script content

Process: Parse with acorn (JS AST parser). Walk AST for danger patterns.

Flags (any one = flagged):

- eval() / new Function() → code execution
- process.spawn / exec / execSync → shell execution
- require('http') / require('https') → outbound network
- require('fs') with write ops → filesystem access
- hex/base64 strings > 50 chars → obfuscation
- require(variable) → dynamic hidden dep

Output: { flagged, patterns, severity, reason }

SIGNAL 3 — Maintainer Signal Scoring

Input: package name, new version

Process: npm registry API fetch. Extract maintainer list, publish dates, signing key. Compare fingerprint between versions.

Flags:

- Signing key fingerprint changed → CRITICAL

- 90+ days inactive then sudden push → HIGH
- New maintainer added same window → HIGH
- Package age < 30 days, high DLs → MEDIUM

Output: { flagged, risk_score (0-100), key_changed, inactive_days, reason }

SIGNAL 4 — Gemini AI Reasoning

Input: combined output of signals 1, 2, 3

Model: gemini-2.5-flash (primary) / gemini-2.5-pro (BLOCK verdicts)

Logic: BLOCK if confidence >= 0.85 and 2+ signals flagged

WARN if confidence >= 0.60 and 1+ signals flagged

PASS otherwise

Output: { verdict, confidence, summary, attack_pattern }

FAIL-SAFE RULES (applied before Gemini, override everything):

- 3+ signals flagged → always BLOCK
- Key changed + new network call → always BLOCK
- Gemini unreachable → rule-based fallback
- Analysis API unreachable → WARN (fail open, never silent block)

COMPLETE API CONTRACT

Base URL: <https://preflight-api.onrender.com>

POST /analyze

Request:

```
{
  "package_name": "axios",
  "old_version": "1.7.9",
  "new_version": "1.7.10",
  "repo": "org/repo-name", // optional
  "pr_number": 42          // optional
}
```

Response:

```
{
  "scan_id": "64a7f3e2b1c4d5e6f7a8b9c0",
  "verdict": "BLOCK",
  "confidence": 0.94,
  "duration_ms": 2840,
}
```

```
"signals": {
  "script_diff": { "flagged": true, "new_hooks": ["postinstall"], "changed_hooks": [], "reason":
"..."},
  "ast_scan": { "flagged": true, "patterns": ["outbound_https", "process_spawn"], "severity":
"high", "reason": "..."},
  "maintainer": { "flagged": true, "risk_score": 92, "key_changed": true, "inactive_days": 238,
"reason": "..."},
  "llm_reasoning": { "verdict": "BLOCK", "confidence": 0.94, "summary": "...", "attack_pattern":
"npm_account_hijack_rat_deployment" }
}
```

GET /health

```
{ "status": "ok", "checks": { "mongodb": "connected", "npm_registry": "reachable", "gemini_api":
"reachable" }, "version": "1.0.0" }
```

GET /scans?page=1&limit=20

```
{ "scans": [ { scan_id, package_name, old_version, new_version, verdict, confidence, repo,
scanned_at, duration_ms } ], "total": 1847, "page": 1, "limit": 20 }
```

GET /scans/:scan_id

Full scan object (same as POST /analyze response + scanned_at)

GET /packages/:name/threat

```
{ package_name, total_scans, block_count, warn_count, pass_count,
community_threat_score, last_flagged_at, flagged_versions, safe_versions }
```

GET /packages/top-threats?limit=10

```
{ "packages": [ { package_name, community_threat_score, block_count, last_flagged_at } ] }
```

MONGODB DATA MODELS

Database: preflight_db

Collection: scans

_id, package_name, old_version, new_version,
verdict ("PASS"|"WARN"|"BLOCK"), confidence (0.0-1.0),
repo (optional), pr_number (optional),

signals { script_diff, ast_scan, maintainer, llm_reasoning },
duration_ms, scanned_at [indexed desc], created_at
Indexes: scanned_at desc · package_name+new_version · verdict

Collection: packages
_id, package_name [unique], total_scans, block_count, warn_count, pass_count,
community_threat_score [indexed desc], last_flagged_at,
flagged_versions [], safe_versions [] (capped at 20), updated_at

Community threat score:
score = (block_count * 100 + warn_count * 40) / total_scans
Capped at 100. Minimum 5 scans before score shown publicly.

GITHUB PR COMMENT FORMAT

🛑 Preflight: BLOCK — Dependency Update Intercepted

`axios` `1.7.9 → 1.7.10` · Confidence: **94%** · 2.84s

> This matches the pattern of a supply chain hijack. New postinstall hook
> with outbound network call combined with signing key rotation after 8
> months of inactivity is high-confidence malicious activity.

Signal	Status	Detail
Script diff	🚨 Flagged	New postinstall hook added
AST scan	🚨 Flagged	Outbound HTTPS + process.spawn detected
Maintainer	🚨 Flagged	Signing key changed, 238 days inactive
Gemini AI	🚨 Flagged	npm account hijack + RAT deployment pattern

Attack pattern: npm_account_hijack_rat_deployment

- What to do:
- ❌ Do NOT merge this PR
 - 🔍 Review axios changelog and npm advisory database manually
 - 🗣️ Report to npm security: security@npmjs.com

[Preflight](https://preflight.dev) · Scan ID: 64a7f3e2b1c4d5e6f7a8b9c0

[View full analysis →](https://preflight.dev/scans/64a7f3e2b1c4d5e6f7a8b9c0)

FRONTEND DESIGN SPEC

AESTHETIC: Neo-Brutalist Terminal

Security tool for developers. Not a SaaS. Not a startup landing page.

Think: war room. High contrast. Raw. Every pixel earns its place.

Hard corners preferred. No soft shadows. No gradients. Flat surfaces.

DESIGN TOKENS:

--bg-primary: #0A0A0A (main background)
--bg-surface: #111111 (card backgrounds)
--bg-elevated: #1A1A1A (hover, active states)
--border: #2A2A2A (default borders)
--border-strong: #404040 (focus, emphasis)
--text-primary: #F0F0F0 (headings, main text)
--text-secondary: #888888 (labels, metadata)
--text-muted: #555555 (placeholders, disabled)
--accent-pass: #00FF88 (PASS — terminal green)
--accent-warn: #FFB800 (WARN — amber)
--accent-block: #FF3B30 (BLOCK — alert red)
--accent-blue: #4A9EFF (links, interactive)
--accent-white: #FFFFFF (high-emphasis)

Typography:

--font-mono: 'JetBrains Mono', monospace (verdicts, IDs, code)
--font-sans: 'Inter', sans-serif (body, UI)
--font-display: 'Space Grotesk', sans-serif (headings, logo)

Spacing (4px base):

--space-1:4px --space-2:8px --space-3:12px --space-4:16px
--space-6:24px --space-8:32px --space-12:48px --space-16:64px

Border radius:

--radius-sm: 4px (badges, pills — use sparingly)
--radius-md: 6px (cards, inputs)
Hard corners everywhere else.

Motion:

--duration-fast: 120ms · --duration-normal: 200ms

--ease-out: cubic-bezier(0.16, 1, 0.3, 1)

PAGES

PAGE 1 — LANDING (/)

Section 1: Hero

Headline (display, large): "The axios attack lasted 3 hours."

Subheadline: "70M weekly downloads. Zero tools caught it."

Body: "Preflight would have blocked it in 30 seconds."

CTA: "Add to your workflow →" (scrolls to install)

Secondary: "See live scans" → /dashboard

Visual: Animated terminal showing a BLOCK verdict appearing on a PR

Section 2: How it works (6-step horizontal stepper)

PR opens → Lockfile diff → Script analysis →

AI reasoning → Verdict posted → PR blocked

Click each step to expand detail.

Section 3: Install snippet

Dark code block, monospace, syntax highlighted.

Copy button. "Copied!" toast on click. Language tag: "yaml".

Section 4: Live community stats (polls API every 10s)

3 large monospace counters:

[X scans run] [Y attacks blocked] [Z packages tracked]

Count-up animation on first viewport entry.

Section 5: The 4 signals (2x2 card grid)

Script Diff · AST Scan · Maintainer · Gemini AI

Each card: name, what it detects, example output snippet.

Footer: Logo | GitHub link | "Powered by Gemini AI + MongoDB Atlas" | MIT

PAGE 2 — DASHBOARD (/dashboard)

Layout: 70% feed / 30% sidebar

Left — Live scan feed:

Header: "Live Scan Feed" + LivePulse (green pulsing dot)
ScanCard per result (collapsed 72px, click to expand):
Row 1: VerdictBadge · package@old→new · repo · timestamp
Row 2: ConfidenceBar · percentage · duration_ms
Row 3: 4 SignalPills (green=clear, red=flagged)
New cards slide in at top. "Load more" pagination below.

Right sidebar:

Panel 1: Top Threats — top 5 by community_threat_score
Panel 2: Today's Stats — scans / blocks / avg confidence
Panel 3: "Try the demo" → /demo

PAGE 3 — DEMO (/demo) ← MOST IMPORTANT FOR THE HACKATHON
Judges and investors run this. No GitHub account needed.

Section 1: Pre-filled locked form

Package: axios · From: 1.7.9 · To: 1.7.10
Label: "The exact attack from March 31, 2026"
Button: "Run Preflight Analysis →" (large, --accent-block bg)

Section 2: Live analysis (appears on button click)

Progress bar fills as signals complete.
4 SignalRows animate in sequentially (150ms stagger):
Script Diff → 🚨 FLAGGED: New postinstall hook added
AST Scan → 🚨 FLAGGED: Outbound HTTPS + process.spawn
Maintainer → 🚨 FLAGGED: Key changed, 238 days inactive
Gemini AI → 🚨 FLAGGED: npm account hijack pattern

Section 3: Verdict card (after all signals complete)

BLOCK (large, monospace, --accent-block, 1px border)
Confidence: 94%
Gemini's full summary paragraph.
"Attack pattern: npm_account_hijack_rat_deployment"
GitHub PR comment preview below.
Button: "View full scan →" → /scans/:id

PAGE 4 — SCAN DETAIL (/scans/:id)

Header: VerdictBadge · axios 1.7.9→1.7.10 · scan_id · 2.84s
Gemini summary (prominent). Confidence bar.
4 expandable signal cards:
Script Diff: status, old/new hooks, actual hook code block
AST Scan: status, patterns, severity, simplified highlight

Maintainer: status, risk score, key change, inactivity, registry link
Gemini: status, full explanation, attack pattern, model used
Footer: Share button | Report false positive

COMPONENT LIBRARY

VerdictBadge props: verdict ("PASS"|"WARN"|"BLOCK")
 monospace, all-caps, 1px border, bg tint per verdict

ScanCard props: scan object
 collapsed 72px · click to expand · hover: border-strong

SignalPill props: name, flagged (bool)
 green dot (clear) / red dot (flagged) + label

ConfidenceBar props: confidence (0.0-1.0)
 4px height · green <0.60 · amber 0.60-0.85 · red >0.85

InstallSnippet props: code, language
 dark block, monospace, copy button, language tag

StatCounter props: value, label
 large monospace number · count-up on viewport entry

SignalRow props: name, status, reason
 pending → analyzing (pulse) → flagged/clear (fade in)

LivePulse CSS only · 8px circle · --accent-pass · 2s pulse loop

REPO STRUCTURE

preflight/
├── preflight-action/ # GitHub Action (TypeScript)

- src/
 - index.ts # Entrypoint: inputs → API → PR comment
 - config/env.ts # Zod-validated env vars
 - adapters/
 - analysis-api.ts # POST /analyze client
 - github.ts # Post PR comment, set status check
 - services/lockfile.ts # Parse package-lock.json diff
 - errors/index.ts
- action.yml
- package.json

- preflight-api/ # FastAPI analysis service (Python)

- app/
 - main.py # App init, lifespan, /health
 - config/settings.py # pydantic-settings
 - routers/
 - analyze.py
 - scans.py
 - packages.py
 - services/
 - script_diff.py # Signal 1
 - ast_scanner.py # Signal 2
 - maintainer.py # Signal 3
 - gemini.py # Signal 4
 - db/
 - client.py
 - scans.py
 - packages.py
 - schemas/analysis.py
 - errors.py
- requirements.txt
- render.yaml

- preflight-web/ # Next.js dashboard (frontend)

- app/
 - page.tsx # /
 - dashboard/page.tsx # /dashboard
 - demo/page.tsx # /demo
 - scans/[id]/page.tsx # /scans/:id
- components/
 - VerdictBadge.tsx
 - ScanCard.tsx
 - SignalPill.tsx

```
| | | — ConfidenceBar.tsx
| | | — InstallSnippet.tsx
| | | — StatCounter.tsx
| | | — SignalRow.tsx
| | | — LivePulse.tsx
| | — lib/api.ts
|
| — demo/ # Verdaccio + mock malicious package
|   | — verdaccio/config.yaml
|   | — mock-axios-malicious/
|
| — CLAUDE.md
| — README.md
```

ENV VARS

preflight-action:

```
PREFLIGHT_API_URL=https://preflight-api.onrender.com
GITHUB_TOKEN= (auto-injected by GitHub Actions)
```

preflight-api:

```
GEMINI_API_KEY=
MONGODB_URI=mongodb+srv://...
MONGODB_DB_NAME=preflight_db
NPM_REGISTRY_URL=https://registry.npmjs.org
ANALYSIS_TIMEOUT_MS=45000
HOST=0.0.0.0
PORT=8000
LOG_LEVEL=INFO
```

preflight-web:

```
NEXT_PUBLIC_API_URL=https://preflight-api.onrender.com
NEXT_PUBLIC_POLL_INTERVAL_MS=10000
```

TECH STACK — FINAL LOCKED

Layer	Technology	Sponsor
Action runtime	GitHub Actions (TypeScript)	GitHub ✓
Analysis API	Python FastAPI	—
API hosting	Render	Render ✓
LLM reasoning	Gemini 2.5 Flash + Pro	Google Cloud + Gemini ✓
Database	MongoDB Atlas	MongoDB ✓
Frontend	Next.js (App Router)	—
Frontend hosting	Vercel (free)	—
AST parsing	acorn	—
npm data	npm Registry REST API	—
GitHub data	GitHub REST API	GitHub ✓
Demo registry	Verdaccio (local)	—

THE 3-MINUTE PITCH SCRIPT

[0:00-0:30] The incident

"On March 31st, 2026, a North Korean state actor hijacked the axios npm account. 70 million weekly downloads. The malicious package was live for 3 hours. npm audit missed it. Snyk missed it. Dependabot missed it. Because the package was legitimately signed and had no CVE."

[0:30-0:50] The gap

"Every existing tool is reactive. They check against known bad packages. Preflight is proactive. It reasons about unknown bad packages — the ones nobody has seen yet."

[0:50-1:30] The demo

"Here's Preflight catching the exact axios attack, live."

- Open /demo, click Run Preflight Analysis
- 4 signals animate in with results
- Verdict: BLOCK — 94% confidence
- Show the PR comment preview

"One line of YAML. That's the entire install. It would have blocked this."

[1:30-2:00] The platform angle

"But Preflight isn't just a tool — it's community threat intelligence.
Every scan stored in MongoDB. The more repos use it,
the smarter it gets for everyone."

→ Switch to /dashboard, show live feed

[2:00-2:30] The stack

"GitHub Action — native to the surface. Gemini 2.5 — explains why, not just what.
MongoDB Atlas — the community brain. Render — live in production.
Open source. MIT. One line to install."

[2:30-3:00] The close

"The axios attack cost teams hours of incident response.
Preflight catches it in 30 seconds, before the code ever runs, for free, forever.
One line in your workflow. Zero compromised dependencies."